

The logo for VIKMO, featuring the word "vikmo" in a bold, black, sans-serif font, centered within a yellow-to-orange gradient square.

AI / ML Intern

Dealer Assistant & Demand Forecasting

Take-Home Assignment

Role	AI / ML Intern — Applied GenAI & Machine Learning
Duration	3 Days
Submission	Public GitHub repository + README + DESIGN.md + eval results
Primary focus	LLM / agent engineering, retrieval, evaluation, ML methodology

1. Introduction

VIKMO's AI layer is conversational and agentic — dealers talk to it in natural language to find parts, check stock, and place orders, and it reasons over a live catalogue and calls tools to act. This assignment mirrors that work. It is deliberately **not** a Kaggle-style modelling exercise: we want to see how you build a reliable LLM-powered assistant, and — separately — whether you have sound classical-ML instincts for a forecasting problem we genuinely care about.

What we are looking for:

- Applied GenAI judgment: retrieval, tool / function calling, grounding, structured output.
- Rigour about reliability: how you evaluate an LLM system and reason about its failures.
- Sound ML methodology for forecasting: clean validation, no leakage, beating a sensible baseline.
- Clean, readable, well-organised code and honest documentation.

Read this first: this take-home earns you a technical round, where you will walk through your code, justify your choices, and modify it live. Build what you understand.

2. Problem Statement

Two parts, over the same auto-parts catalogue:

- **Part A — Dealer Assistant (core):** a conversational assistant that helps a dealer find parts, check stock, and place orders, grounded in the catalogue and able to call tools to act.
- **Part B — Demand Forecasting (bonus):** given historical sales, forecast near-term demand per product and beat a naive baseline.

Part A is the core of the evaluation. Part B is a **weighted bonus**.

3. Technical Requirements

Component	Requirement
Language	Python
LLM	Any provider. We recommend Google Gemini's free tier (mirrors our stack, costs nothing). OpenAI, Groq, or a local model (e.g. Ollama) are all fine.
Frameworks	Your choice — LangChain, LlamaIndex, the provider SDK directly, or your own orchestration. Use what you can explain.
Secrets	Do not hardcode API keys; read them from environment variables.

4. Provided Data

- **Product catalogue:** ~500–1,000 auto-parts SKUs (CSV / JSON) with fields such as SKU, name, category, price, stock, and vehicle fitment. Used by Part A.
- **Sales history:** a CSV of historical sales (date, SKU, units sold, promo flag), ~30 SKUs over 12–24 months, with trend, seasonality, and occasional promo spikes. Used by Part B.

5. Part A — Dealer Assistant (core)

Build a conversational assistant over the catalogue. It must satisfy the following.

5.1 Retrieval (RAG)

Index the catalogue and retrieve the relevant products for a user's query. The catalogue is large enough that stuffing the whole thing into the prompt is **not** a valid approach — implement real retrieval (embeddings + vector search, or a justified alternative). Explain your chunking / embedding / indexing choices in DESIGN.md.

5.2 Tools (function calling)

The model must call tools to act, choosing them from the conversation. Implement at least:

- **check_stock** — look up availability for a product.
- **create_order** — place an order for a dealer with line items, returning a structured result.
- **find_parts_by_vehicle** — find parts that fit a given make / model / year.

Order creation must use **structured output** (e.g. validated JSON) — not free text.

5.3 Conversation

- Handle multi-turn dialogue and maintain context across turns.
- Ask clarifying questions when a request is ambiguous (e.g. *"I need brake pads"* → *"for which bike?"*).

5.4 Grounding

- Answers about products, prices, and stock must come from the data, not be invented.

5.5 Evaluation (carries real weight)

- Provide an eval set: test interactions covering happy paths, tricky / ambiguous queries, and out-of-scope requests.

- Define what 'correct' means for each, run your assistant against the set, and report results with simple metrics.
- **Analyse failure modes honestly** — where it breaks, why, and what you would change. This, not a polished demo, is the clearest signal of an AI engineer to us.

Interface: a CLI or script is fine. A chat / WhatsApp-style UI is a bonus.

6. Part B — Demand Forecasting (weighted bonus)

Using the provided sales history, forecast near-term demand (e.g. the next 4 weeks) per SKU.

- **Hold out the most recent period as a test window**; fit only on data before it — no leakage from the future into training.
- Report error per SKU and overall using a sensible metric (e.g. MAE or MAPE).
- **Beat a naive baseline** (e.g. last-value, seasonal-naive, or a moving average). Report the baseline's error and show your approach beats it.
- Justify any complexity over the baseline — if a simple method wins, say so.

Document your approach, validation scheme, and how you prevented leakage in DESIGN.md.

7. DESIGN.md (required)

Explain your key decisions. We grade the reasoning and will ask you to defend it:

- Retrieval approach and why; embedding / indexing choices.
- Tool design and how the model decides to call them.
- Prompt design and any guardrails against hallucination or off-topic use.
- Evaluation methodology and what the failures taught you.
- (Part B) Model choice, validation scheme, and leakage prevention.

8. Evaluation

Total: **100 points** for the core (Part A). Bonuses are added on top.

Criteria	Points	What we check
Retrieval / RAG	20	Real retrieval, sensible indexing, scales beyond prompt-stuffing
Agent & Tool-Calling	25	Tool design, correct invocation, structured order output
Conversation Handling	10	Multi-turn context, clarifying questions
Evaluation & Failure Analysis	20	Meaningful eval set, metrics, honest failure analysis
Code Quality	10	Clean, readable, well-structured
DESIGN.md & Documentation	15	Setup, clear reasoning, methodology

Bonus points

Bonus item	Extra
Demand Forecasting (Part B) — beats baseline, leakage-free backtest, clear metrics	+15–20
Chat / WhatsApp-style UI	+5
Multimodal — identify a part from an image	+5
Guardrails against off-topic / hallucination	+5

9. Submission

9.1 Repository structure

```

your-repo/
├── README.md
├── DESIGN.md
├── requirements.txt
├── assistant/          (retrieval, tools, agent loop)
├── eval/              (eval set + results)
└── forecasting/       (optional – Part B)

```

9.2 README must include

1. Overview and what you implemented.
2. Tech stack and which LLM / provider you used.
3. Setup (environment variables, how to run the assistant and the eval).
4. Example interactions.
5. Any assumptions, with a pointer to DESIGN.md and your eval results.

9.3 Submission checklist

- Code pushed to a public GitHub repository.
- README and DESIGN.md complete and clear.
- The assistant runs; the eval set and its results are included.
- If attempted, forecasting code and results are included.
- No hardcoded API keys or secrets.

10. Sample Interactions (Part A)

Your assistant should handle interactions like these:

- *"Do you have brake pads for a Bajaj Pulsar 150?"* → retrieves fitting parts, confirms the vehicle if needed, returns options with price and stock.
- *"Place an order for 10 units of [SKU] for ABC Motors."* → calls `create_order` with a structured payload and returns a confirmation.
- *"What's the cheapest chain lube you stock?"* → a grounded answer from the catalogue.
- *"I need tyres."* → asks a clarifying question (vehicle / size) before answering.
- *"What's the weather today?"* → stays on-domain / politely declines (guardrail).

Note on AI Usage

You may use AI tools (Claude, ChatGPT, GitHub Copilot, and similar) for coding — we use them too. There is one rule: **you must fully understand everything you submit**. This take-home only earns you a technical round, where we will ask you to walk through your code, explain your design decisions, and modify your implementation on the spot. AI used without understanding will be evident and will not help you.

Questions?

If anything about the requirements is unclear, ask before you start — we would much rather clarify than have you guess. Good luck; we look forward to reviewing your work.

— Team VIKMO